Generación de Números Aleatorios

Estructuras de Control, Ciclos y Condicionales ($\mathcal{C}^3$) III · Caminatas aleatorias y difusión

Carlos Pinilla

pinilla.carlos@correounivalle.edu.co

Basado en parte en material de A. Argüelles y C. Galíndez

Contenido

1. Números pseudoaleatorios: uniformidad e independencia.
2. Métodos de generación: físicos, cuadrados medios, congruenciales, Fibonacci, Mersenne Twister y caos.
3. Pruebas de calidad de un generador.
4. Aplicación física: caminatas aleatorias y difusión.
5. Taller 4: coeficiente de difusión.

Continuidad con la Semana 3

Semana 3 – Fortran

- Ciclos do y acumuladores.
- Sumas de mínimos cuadrados.
- Ajuste de datos a un modelo.

Semana 4 – Python

- Números pseudoaleatorios.
- Ciclos que simulan azar.
- El mismo ajuste, ahora para medir difusión.

Los acumuladores y el ajuste lineal de la Semana 3 se reutilizan.

Uniformidad e independencia

Uniforme

Los valores se reparten por igual en el intervalo.

Ejemplo: $1, 2, 3, 4, \ldots$ es uniforme, pero no es aleatoria.

Independiente

Conocer un valor no dice nada del siguiente.

Ejemplo: $r_1,\ 1 - r_1,\ r_2,\ 1 - r_2,\ \ldots$ parece aleatoria, pero cada par está correlacionado.

Requisito para la simulación

En una simulación física, las correlaciones ocultas producen resultados equivocados aunque la distribución se vea uniforme.

Números pseudoaleatorios y semilla

- Un computador es determinista: sus “números aleatorios” salen de una fórmula.
- Se llaman **pseudoaleatorios**: parecen azar, pero la misma **semilla** produce la misma secuencia.

```python
import random

rng1 = random.Random(42)     # dos generadores con la misma semilla
rng2 = random.Random(42)
print(rng1.random() == rng2.random())    # True
```

Reproducibilidad

Con la semilla fija, una simulación se puede repetir exactamente para depurarla y verificarla.

Propiedades de un buen generador

1. **Patrón aleatorio**: pasa pruebas estadísticas.
2. **Período largo**: tarda mucho en repetirse.
3. **Repetibilidad**: la misma semilla da la misma secuencia.
4. **Eficiencia**: es rápido y usa poca memoria.

Fuente: M. T. Heath, Scientific Computing, cap. 13.

Panorama de los métodos de generación

Método	**Idea**	**Ventaja**	**Limitación**
Físicos	Medir un proceso físico aleatorio (ruido térmico, decaimiento)	Aleatoriedad verdadera	Lentos, no reproducibles
Cuadrados medios (1946)	Elevar al cuadrado y tomar los dígitos centrales	Muy simple	Ciclos cortos; puede caer en 0
Lineal congruente	$r_{i+1} = (ar_i + c)$ mód M	Rápido; un solo número de estado	Período $\leq M$; correlaciones
Fibonacci retardado	$x_k = x_{k-q} \pm x_{k-p}$	Período muy largo; sin divisiones	Requiere memoria e inicialización
Mersenne Twister (1997)	624 enteros mezclados con operaciones de bits	Período $2^{19937} - 1$	Predecible: no apto para criptografía
Caos y autómatas	Iterar un sistema dinámico caótico	Ilustra el paso de determinismo a azar	Distribución no uniforme; lentos

En orden histórico: de la física al algoritmo. Python y NumPy usan generadores algorítmicos.

Generadores físicos

- Se mide un proceso intrínsecamente aleatorio:
  - tiempos entre cuentas de un contador Geiger (decaimiento radiactivo);
  - ruido térmico (Johnson) en una resistencia;
  - fotones en un divisor de haz.
- 1955: la corporación RAND publicó el libro *A Million Random Digits*, generado con una ruleta electrónica.
- Hoy: los procesadores incluyen fuentes de ruido térmico, usadas por el sistema operativo para criptografía.

Ventaja

Aleatoriedad verdadera: el resultado no se puede predecir.

Limitaciones para simular

- No son reproducibles: no se puede repetir una simulación.
- Son lentos frente a un algoritmo.
- Presentan sesgos que deben corregirse.

Método de los cuadrados medios (von Neumann, 1946)

Algoritmo (4 dígitos)

1. Elevar x_i al cuadrado: 8 dígitos.
2. Tomar los 4 dígitos centrales: x_{i+1}.

x_i	x_i^2	x_{i+1}
1234	01**5227**56	5227
5227	27**3215**29	3215
3215	10**3362**25	3362
⋮	⋮	⋮

Qué ocurre después:

- Semilla 1234: tras 56 pasos llega a **0** y se queda ahí.
- Semilla 2100: ciclo de 4 valores (2100, 4100, 8100, 6100).
- Semilla 3792: $3792^2 = 14\,\mathbf{3792}\,64$, punto fijo.
- Con 4 dígitos, ninguna semilla produce más de 111 valores distintos, de 10 000 posibles.

Lección histórica

Un procedimiento complicado no garantiza aleatoriedad.

Generador lineal congruente (LCG)

Recurrencia

$$r_{i+1} = (a\, r_i + c) \bmod M, \qquad x_i = \frac{r_i}{M} \in [0, 1)$$

a: multiplicador c: incremento M: módulo r_1: semilla

Ejemplo a mano con $a = 4$, $c = 1$, $M = 9$, $r_1 = 3$:

i	1	2	3	4	5	6	7	8	9	10
r_i	3	4	8	6	7	2	0	1	5	**3**
$x_i = r_i/9$	.333	.444	.889	.667	.778	.222	.000	.111	.556	**.333**

Al reaparecer el 3, toda la secuencia se repite: el período es 9. Dividir por M no alarga el período.

Cambio de intervalo: de $[0, 1)$ a $[A, B)$

Transformación lineal

$$y_i = A + (B - A)\, x_i, \qquad x_i \in [0, 1)$$

produce números uniformes en $[A, B)$.

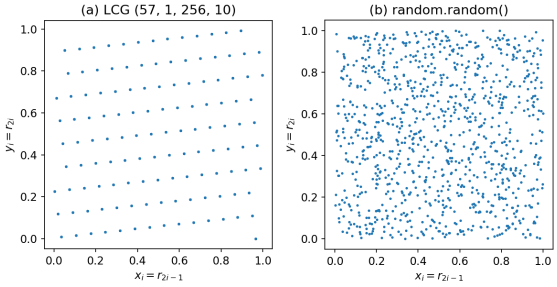
Ejercicio: escalar la secuencia anterior al intervalo $[10, 20)$.

r_i	x_i	$y_i = 10 + 10\, x_i$
3	0.333	13.33
4	0.444	14.44
8	0.889	18.89
6	0.667	16.67
7	0.778	17.78
⋮	⋮	⋮

Uso físico: posiciones iniciales en una caja, velocidades en un intervalo, pasos de una caminata en $[-1, 1)$.

Período y elección de parámetros

- El período nunca supera M: se necesita M grande.
- Pero un período largo no basta.
- El LCG $(57, 1, 256)$ recorre los 256 valores posibles... y sus pares consecutivos caen en una rejilla.



Recomendación

Graficar pares (x_i, x_{i+1}) sin unir los puntos.

Parámetros típicos y desbordamiento (*overflow*)

Valores típicos de M

- Enteros de 32 bits: $M = 2^{31} \approx 2 \times 10^9$.
- Enteros de 48 bits: $M = 2^{48} \approx 3 \times 10^{14}$.

Ejemplo: `drand48` (biblioteca de C)

$$a = \mathrm{5DEECE66D}_{16} \approx 2{,}5 \times 10^{10}$$

$$c = \mathrm{B}_{16} = 11, \qquad M = 2^{48}$$

Desbordamiento

El producto $a\, r_i + c$ debe caber en un entero de la máquina. Con `drand48`:

$$a(M-1) + c \approx 7 \times 10^{24} \approx 2^{83}$$

que supera ampliamente el máximo de 64 bits ($2^{63} \approx 9 \times 10^{18}$).

En Python no hay problema: sus enteros crecen sin límite. En Fortran, con `integer(8)`, sí lo habría.

Ejercicio: período y correlaciones del LCG $(57, 1, 256)$

1. Programar el LCG con $(a, c, M, r_1) = (57, 1, 256, 10)$.
2. Contar con un ciclo cuántos números salen antes de que se repita.
   Respuesta: 256, el máximo posible.
3. Graficar los pares (x_i, x_{i+1}): aparecen en unas pocas líneas paralelas.
4. Repetir con los parámetros de `drand48` y con `random.random()`: los pares llenan el cuadrado.

Conclusión

Período máximo no significa buena calidad. La elección de a y c es crucial.

Generadores de Fibonacci retardados

Idea

Cada número se forma con dos valores *anteriores* de la secuencia, con retardos $p < q$:

$$x_k = (x_{k-q} \pm x_{k-p}) \text{ mód } M$$

Ejemplo de juguete: sucesión de Fibonacci módulo 10 ($q = 2$, $p = 1$):

1, 1, 2, 3, 5, 8, 3, 1, 4, 5, 9, 4, 3, 7, 0, 7, ...

Período 60, aunque solo hay 10 dígitos: el estado es el par de valores anteriores.

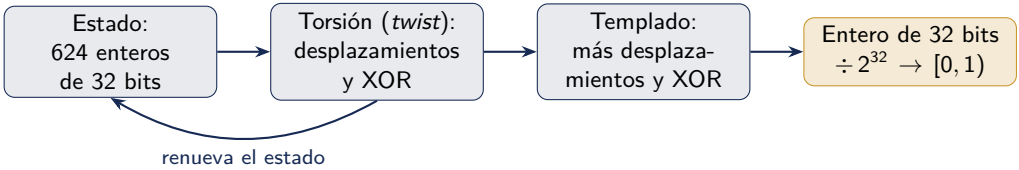
Versión práctica (Heath, cap. 13):

$$x_k = x_{k-17} - x_{k-5}, \qquad x_k \in [0, 1)$$

si el resultado es negativo, se suma 1.

- No necesita multiplicaciones ni divisiones.
- Repetir un valor no obliga a repetir la secuencia: períodos mucho más largos que M.
- Requiere guardar los últimos 17 valores y generarlos al inicio con otro método.

Mersenne Twister: funcionamiento



- El estado ocupa $624 \times 32 = 19\,968$ bits, frente a un solo número en el LCG.
- Período $2^{19937} - 1$, un primo de Mersenne: de ahí el nombre.
- Uniforme en hasta 623 dimensiones.

Limitación

Es un método lineal: observar 624 salidas seguidas permite reconstruir el estado y predecir todo lo que sigue. Para criptografía se usa el módulo `secrets`.

Generadores incorporados en Python

- `random` usa Mersenne Twister (Matsumoto y Nishimura, 1997); `numpy.random.default_rng` usa PCG64, un generador más reciente.
- Regla práctica: usar el generador de la biblioteca, no uno propio.

```python
import random

rng = random.Random(2026)     # generador propio con semilla fija

u = rng.random()              # real uniforme en [0, 1)
v = rng.uniform(10, 20)       # real uniforme en [10, 20)
n = rng.randint(1, 6)         # entero en {1, ..., 6}: un dado
```

Para escalar a otro intervalo: $x = A + (B - A)\,u$.

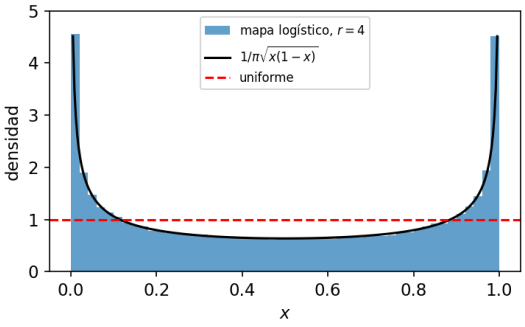
Caos determinista: el mapa logístico

- El mapa logístico

$$x_{n+1} = 4\,x_n(1 - x_n)$$

es caótico: su secuencia parece aleatoria.

- Pero sus valores se acumulan en los bordes: **no es uniforme**.
- Media $\approx 0{,}5$ (como la uniforme), pero varianza $\approx 0{,}125$ en lugar de $1/12 \approx 0{,}083$.



Conclusión

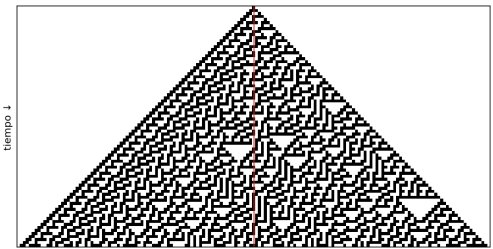
"Caótico" no significa "buen generador".
Hay que probarlo.

Autómata celular: la Regla 30 (Wolfram, 1983)

- Una fila de celdas con valor 0 o 1.
- En cada paso, cada celda se actualiza con sus dos vecinas:

$$\text{nueva} = \text{izq.} \oplus (\text{centro} \vee \text{der.})$$

- Los 8 vecindarios posibles producen $00011110_2 = 30$: de ahí el nombre.
- La columna central, $1101110011000101\ldots$, se usó como generador de números en *Mathematica*.



Un patrón similar aparece en la concha del caracol marino Conus textile.

Pruebas de uniformidad e independencia

Uniformidad: momentos

$$\langle x^k \rangle = \frac{1}{N} \sum_{i=1}^{N} x_i^k \approx \frac{1}{k+1}$$

$\langle x \rangle \approx \frac{1}{2}, \quad \langle x^3 \rangle \approx \frac{1}{4}, \quad \langle x^7 \rangle \approx \frac{1}{8}$

Independencia: correlación entre vecinos

$$C(1) = \frac{1}{N} \sum_{i=1}^{N} x_i \, x_{i+1} \approx \frac{1}{4}$$

si los números consecutivos no están correlacionados.

- La desviación respecto al valor esperado debe disminuir como $1/\sqrt{N}$.
- Ninguna prueba certifica aleatoriedad: solo permite detectar generadores deficientes.

Cálculo de las pruebas con acumuladores

```python
import random

# --- Generador, tamaño de la muestra y acumuladores ---
rng = random.Random(1)
N = 100_000
s1 = s3 = c1 = 0.0
x_ant = rng.random()              # valor anterior, para C(1)

# --- Un número por iteración; se acumula, no se guarda ---
for i in range(N):
    x = rng.random()
    s1 += x                       # para <x>
    s3 += x**3                    # para <x^3>
    c1 += x_ant * x               # para C(1)
    x_ant = x

print(s1/N, s3/N, c1/N)           # salida: 0.5002 0.2505 0.2500
```

Igual que las sumatorias de mínimos cuadrados de la semana 3: sin arreglos, y fácil de escribir en Fortran.

Comparación de tres generadores

$N = 100\,000$ números de cada generador:

Generador	$\langle x \rangle$	$\langle x^3 \rangle$	$C(1)$	$\sqrt{N}\,\lvert\langle x \rangle - \frac{1}{2}\rvert$
Esperado	0.5	0.25	0.25	del orden de 1
`random.Random`	0.5002	0.2505	0.2500	0.06
LCG `drand48`	0.5006	0.2505	0.2505	0.20
LCG $(57, 1, 256)$	0.4981	0.2481	**0.2561**	0.61

- El LCG deficiente **pasa** la prueba de momentos…
- …pero **falla** la de vecinos: se desvía 0,006, cuando la fluctuación esperada es $\approx 0{,}0007$.

Una sola prueba no basta: los momentos miden uniformidad; $C(1)$ mide independencia.

Caminata aleatoria como modelo de difusión

- Una gota de tinta en agua en reposo se dispersa sin agitación externa.
- Cada partícula choca millones de veces con las moléculas de agua.
- Se modela cada partícula como un **caminante aleatorio**.

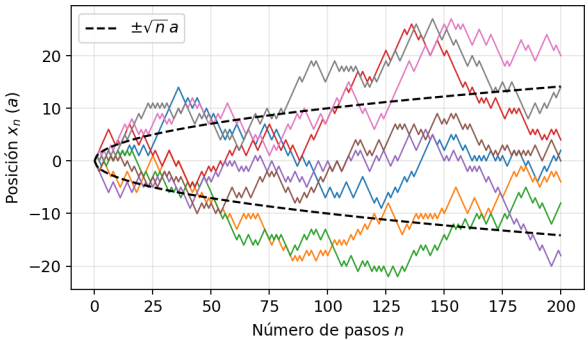
Modelo 1D

En cada paso se genera $u \in [0, 1)$:

$$s_i = \begin{cases} +a & u < 0{,}5 \\ -a & u \geq 0{,}5 \end{cases}$$

$$x_n = s_1 + s_2 + \cdots + s_n$$

Trayectorias individuales y promedio de conjunto



- Cada trayectoria individual es irregular.
- El conjunto se ensancha de forma ordenada, como $\sqrt{n}\,a$.
- Para medirlo hay que **promediar sobre muchos caminantes**.

Desplazamiento cuadrático medio

Con $\langle s_i \rangle = 0$ y $\langle s_i^2 \rangle = a^2$:

$$\langle x_n \rangle = 0, \qquad \langle x_n^2 \rangle = \sum_i \langle s_i^2 \rangle + 2 \sum_{i<j} \underbrace{\langle s_i s_j \rangle}_{=0} = n\, a^2$$

Papel del generador

Los términos cruzados se anulan **solo si los pasos son independientes**. Un generador con correlaciones cambia el resultado físico.

Nota: $\langle x_n^2 \rangle \neq \langle x_n \rangle^2$. La media es cero, pero la dispersión crece.

Coeficiente de difusión

- La difusión en 1D cumple $\langle x^2 \rangle = 2Dt$.
- D es el **coeficiente de difusión**: qué tan rápido se dispersan las partículas. Unidades: m^2/s.
- Con $t = n\,\Delta t$ y $\langle x_n^2 \rangle = na^2$:

$$\boxed{D = \frac{a^2}{2\,\Delta t}}$$

Consecuencia

$t \propto L^2$: dispersarse **el doble de distancia** toma **cuatro veces más tiempo**.

En d dimensiones

$\langle r^2 \rangle = 2dDt$. Una caminata 2D con pasos continuos en $[-1, 1)$ da $\langle R^2 \rangle = \frac{2}{3}\,n$.

Simulación de K caminantes con acumuladores

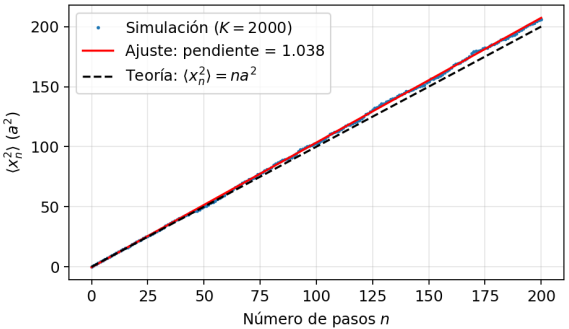
```python
import random

# --- Parámetros y acumuladores (uno por instante n) ---
K, N, a = 2000, 200, 1.0
rng = random.Random(2026)      # una semilla para todo
suma_x2 = [0.0]*(N + 1)        # suma de x_n^2

# --- Externo: caminantes; interno: pasos ---
for caminante in range(K):
    x = 0.0                    # parte del origen
    for n in range(1, N + 1):
        x += a if rng.random() < 0.5 else -a  # paso
        suma_x2[n] += x*x      # acumula x_n^2

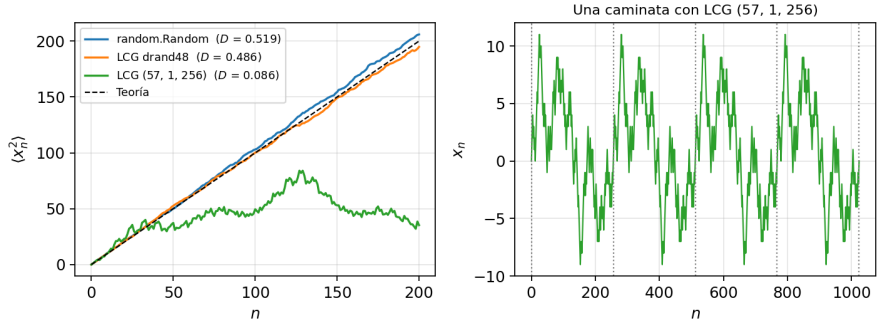
# --- Promedio: dividir por K al final ---
msd = [s/K for s in suma_x2]  # <x_n^2>
```

Estimación de D por mínimos cuadrados



- Se ajusta $\langle x_n^2 \rangle = m\,n + b$.
- $D_{\text{est}} = m/(2\Delta t)$.
- Resultado: $D_{\text{est}} = 0{,}519$, frente a $D = 0{,}5$.
- La diferencia es fluctuación estadística.

Efecto de un generador deficiente sobre D



Con el LCG $(57, 1, 256)$: $D = 0{,}086$ en lugar de 0,5

Su período es 256 y exactamente la mitad de sus valores es menor que 0.5: la caminata vuelve al origen cada 256 pasos. No hay difusión.

Escalas de tiempo de la difusión

Oxígeno disuelto en agua:

$$D \approx 2 \times 10^{-9}\ \mathrm{m}^2/\mathrm{s}, \qquad t \approx \frac{L^2}{2D}$$

Distancia	Tiempo
10 μm (una célula)	0,025 s
1 mm	4 min
1 cm	7 h
1 m	8 años

Interpretación física

La difusión es eficiente a escala celular, pero muy lenta a escala macroscópica.

Con $\Delta t = 1\ \mu$s, el paso es $a = \sqrt{2D\Delta t} \approx 63$ nm. Simular 1 cm exigiría $\sim 2{,}5 \times 10^{10}$ pasos: a gran escala se usa la ecuación de difusión.

Pregunta para discusión

Si la difusión tarda años en recorrer un metro, ¿qué mezcla un contaminante en un lago?

Taller 4: coeficiente de difusión a partir de caminatas aleatorias

- **A** Funciones de la simulación, con ciclos y acumuladores propios.
- **B** Ajuste lineal con sumas explícitas.
- **C** Estimar el coeficiente de difusión y compararlo con $a^2/2\Delta t$.
- **D** Repetir con un generador deficiente y comparar el coeficiente obtenido.
- **E** Llevar el resultado a unidades reales.
- **F** Informe de una página.

La última celda del notebook verifica el trabajo con pruebas PASS/FAIL.

Conclusiones

- Un buen generador debe ser uniforme **e** independiente, y eso se prueba, no se supone.
- La semilla hace que una simulación con azar sea reproducible.
- Una caminata aleatoria reproduce la difusión: $\langle x^2 \rangle = 2Dt$.
- Un generador deficiente puede alterar un resultado físico en un factor seis.

Próxima semana: cálculo de π por el método de Monte Carlo.